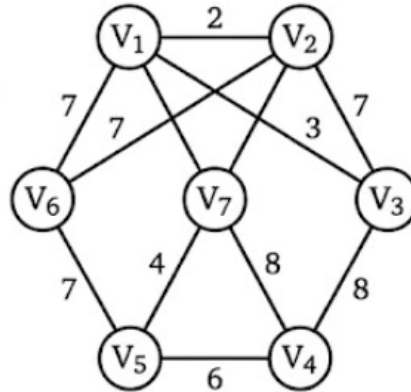


Week 08: Minimum spanning tree

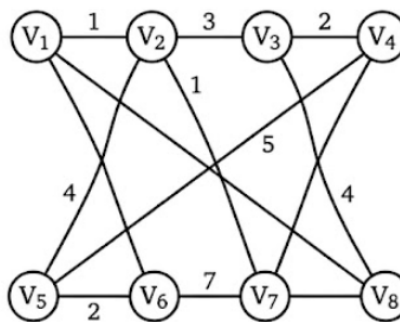
1 Exercises

Exercise 1. Given this graph, get the minimum spanning tree using:



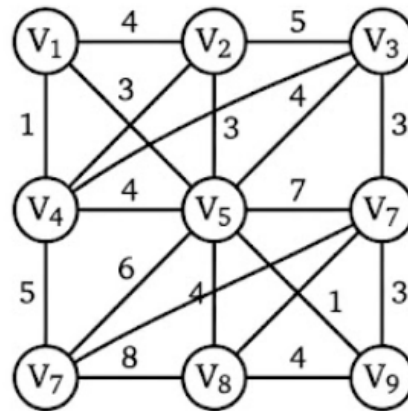
- Prim's Algorithm starting with the vertex V_7 .
- Kruskal's Algorithm.
- At which step the two algorithms pick the same edge for the first time. Do they always result in the exact same set of edges?

Exercise 2. Given this graph, suppose the weight of one edge already in the MST increases by X , and the weight of one edge not in the MST decreases by Y :



- What is the minimum value of Y required to force the MST to change?
- If an edge weight increases, does it always stay in the MST as long as it is still the cheapest way to connect a specific vertex? Explain?

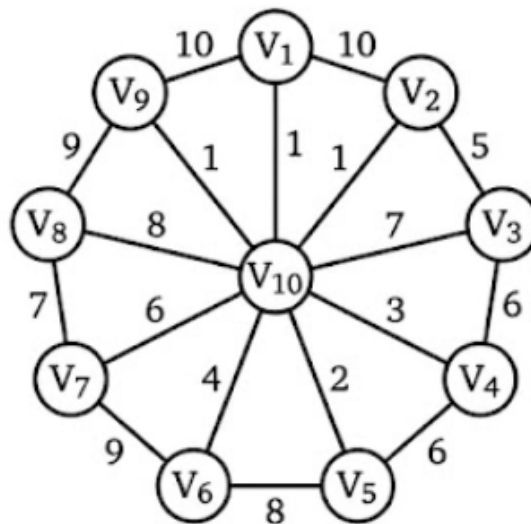
Exercise 3. Given this graph:



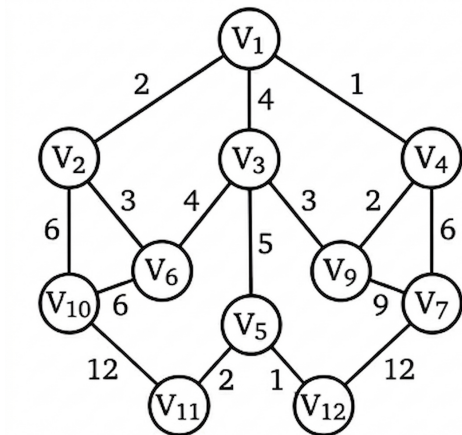
- Prove and demonstrate via the this graph that the maximum weight edge on the MST path between any nodes u and v is the minimum possible "bottleneck" weight among all possible paths between those two nodes in the original graph.
- Why is this property useful for designing a communication network that needs to guarantee a minimum bandwidth?

Exercise 4. Given this graph, run Prim's algorithm under the following constraints and determine if the resulting tree is still technically a "Minimum" Spanning Tree for the remaining available graph:

- **Constraint 1:** You must include the edge (V_1, V_{10}) in your spanning tree.
- **Constraint 2:** You are forbidden from using the cheapest edge.



Exercise 5. Given this graph:



- Modify Kruskal's algorithm to find a Maximum Spanning Tree (the most expensive way to connect all nodes without cycles).
- If every single edge in this 11-vertex graph had a unique weight (no two edges are the same), would the MST be unique?
- If you add 1 to every edge weight in the graph, does the MST change? What if you multiply every edge weight by 2? Justify your answer.

2 Programming

Given connected graph G :

Table 1: Adjacency List of G

Source (u)	Adjacency List (Destination, Weight)
1	(2, 3), (3, 8)
2	(1, 3), (4, 7), (5, 5)
3	(1, 8), (4, 2), (6, 10)
4	(2, 7), (3, 2), (5, 1)
5	(2, 5), (4, 1), (7, 4)
6	(3, 10), (7, 6), (8, 3)
7	(5, 4), (6, 6), (9, 9)
8	(6, 3), (9, 5), (10, 1)
9	(7, 9), (8, 5), (10, 7)
10	(8, 1), (9, 7)

Bài 1. Read file and implement the class represents the graph using adjacency matrix và Edge List.

Bài 2. Write the functions to perform these tasks:

- BFS search.

- b) DFS search.
- c) Count connected components.

Bài 3. Write the functions to find the shortest path of the node pair (Start,End):

- a) Using BFS (non-weight)
- b) Dijkstra's (using weight)
- c) Bellman–Ford và Floyd–Warshall (negative weight)